

ha.ckers.org web application security lab - Archive » CSS History Stealing Acts As Cookie

ha.ckers.org web application security lab - Archive » CSS History Stealing Acts As Cookie -

Author not stated, ha.ckers.org.

- Published: date not stated
- Original: <<http://ha.ckers.org/blog/20060823/css-history-stealing-acts-as-cookie/>>
- Preserved from: <http://ha.ckers.org/blog/20060823/css-history-stealing-acts-as-cookie/> (stored on 2026-08-09)
- Capture timestamp: 20071002223204
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

ha.ckers.org web application security lab - Archive » CSS History Stealing Acts As Cookie

[[image: image]](<http://www.webappsec.org/>) Paid Advertising [[image: web application security lab]](<http://ha.ckers.org/>)

CSS History Stealing Acts As Cookie

Matan, Jeremiah and I have been chatting a bit lately around the [CSS history stealing hack that Jeremiah came up with](#) and presented at his Blackhat talk a few weeks ago. One of the ideas Matan came up with I think is worth posting, because it really does show a rather interesting application for CSS history stealing that I hadn't thought about before.

In its simplest form, once a user visits a certain web page an attacker could choose a random unique id for the user and then force him to visit a URL (through a hidden iframe) containing that unique ID. Then on a subsequent visit the attacker can make the user iterate through all the unique ids that it ever generated and see if the user visited any of them. If he did, then the attacker can know the user has already been to his site. Furthermore, the URL with the unique id can point to a script that stores and retrieves the data the attacker would like to save for the user. Of course, in the real world this wouldn't work well because as the number of visitors increases, so do the number of URLs a user would have to iterate through. When it comes to thousands or millions of users this could take quite a while. To solve this, an attacker can use a hierarchy of

folders (or most likely virtual folders that don't even exist). So let's say you have a special folder /spy/ on the web server. This folder contains numbered subfolders 0-9. Each of these folders would contain 0-9 more subfolders. The nesting of the folders could be around 10 folders deep which means the site can hold 10^{10} unique ids. Then the attacker can generate a random path for the user and force him to visit each of the folders in the path (so the final path would look like /spy/3/6/8/1/7/2/3/4/8/9). In this example it would take 10 URL accesses to reach the end of the path. So once a user enters the site again he would have to iterate through a maximum of $10 * 10$ links in a worst case scenario. Of course, security-wise for the attacking site this is bad as random users would be able to impersonate other random users and mangle their stored data. But this can be overcome by generating valid paths using an algorithm only the website knows and most complete paths will lead to a dead end. Do you think this could work?

Yes, yes, it very easily could. This is actually an interesting way to get around some of the issues really large sites have with companies like AOL that have massive super proxies with upwards of 30k people behind a single IP address. You can "cookie" them in this way with a relatively small footprint compared to an actual cookie (which is often killed anyway by security products or turned off entirely by paranoid/malicious users) and upon a user repeat visit you can detect them once again. It'll be interesting to see how this attack evolves over time, as I am sure there are dozens of other interesting ways to use these attacks. Special thanks to [Matan!](#)

This entry was posted on Wednesday, August 23rd, 2006 at 8:29 pm and is filed under [Webappsec](#). You can follow any responses to this entry through the [RSS 2.0](#) feed. You can leave a response, or [trackback](#) from your own site.

Leave a Reply Or Discuss [On the Forums](#)

Name (required)

Mail (will not be published) (required)

Website

Archived from <http://ha.ckers.org/blog/20060823/css-history-stealing-acts-as-cookie/>. Rendered offline from the local Markdown copy.